

Exact Ideals and Optimal Jumps

in Coarse Description Classes

Countable ideal realization, fixed-oracle exact partners,
and a uniform classification of the guarded diagonal set

Prepared for Vladimir Reshetnikov

28 September 2026

Abstract

A coarse description of a set agrees with it outside a set of asymptotic density zero. Its common-information core consists of the sets computed by every such description. We prove that the possible cores are exactly the countable Turing ideals, completing target C4 of the inspected ProveIt research plan. We establish a layered-reservoir theorem that simultaneously realizes a prescribed ideal as a common lower cone, avoids a fixed oracle, controls intersections of finite joins, and bounds the jump of an entire countable join. If the presentation is computable in an oracle H , the construction gives a join Z with $(Z \oplus H)' \equiv_T H'$ and $\mathcal{L}(Z) \cap \mathcal{L}(H) = I$.

For the plan's guarded many-one complete c.e. set X , we prove the uniform coarse equivalence $X \equiv_{uc} \mathcal{R}(\text{TOT})$. Consequently its description spectrum is exactly $\{\mathbf{d} : \mathbf{0}'' \leq \mathbf{d}'\}$. Combining this classification with the reservoir construction produces descriptions $D_i \approx X$ which form minimal pairs with $\emptyset'$ and satisfy, for $Z = \bigoplus_i D_i$,

$$D_i' \equiv_T Z' \equiv_T (Z \oplus \emptyset')' \equiv_T \emptyset''.$$

This resolves the prescribed-c.e.-set component of C6, including its residual first-jump question, with an optimal first-jump bound. The proofs distinguish relative lowness from ordinary lowness and preserve overlap when analyzing finite joins. We finish with precise remaining questions and a formalization plan.

Status: a conventional mathematical research manuscript, not a proof-assistant certificate. The repository already supplies the guarded set, the dyadic coding criterion, reservoir methods, and a sketch of C4. The extensions proved here are identified separately. Bibliographic priority is not established; no claim to resolve a longstanding global conjecture is made.

Contents

1	Targets, contributions, and the status of the results	2
2	Preliminaries: density, information cores, and coding	5
3	The guarded diagonal set has exactly the high spectrum	9
4	Layered reservoirs and the finite-extension lemmas	11
5	The countable-array exact-ideal theorem	14
6	Every countable Turing ideal is a core: target C4	17
7	Optimal exact partners of the guarded set: target C6	19
8	Examples, presentation dependence, and boundaries	20
9	Verification, effective content, and formalization	22
10	Further research questions and a prioritized programme	24
11	Conclusion	25
A	An explicit proof audit	26
B	Source provenance and reproducibility	27

[write] **Research report 13 of the CoarseDegrees programme** (batch 37, manuscript 09). Text marked [write] was added when it was filed; Sections 1.4–1.7 give the notation against the plan and the other round-2 reports, the provenance, the formal status and the plan targets addressed. Nothing in this report is formalized. It is AI-assisted and unrefereed.

1 Targets, contributions, and the status of the results

The question is not merely how complicated one representative of a coarse class can be. Sparse coding makes high-complexity representatives easy to obtain. The more structural question is how little information two representatives can share while remaining equal on a density-one set, and how much jump complexity is required to obtain such witnesses.

This article examines the computability research in the Provelt repository with source files verified at commit

6c9175a54bcaad3bfc2d416257d2b7d3dff2c1e0.

The research plan and its round-one synthesis are the immediate sources of the targets [5, 6]. The plan explicitly treats several proposals as unreviewed. We therefore prove the assertions used here rather than taking an unverified existence theorem from the repository as an axiom.

1.1 The two targets addressed

C4: realization of countable ideals. The plan proposes a column-wise robust code for a sequence (A_i) and conjectures that its core is the ideal generated by the finite joins of the A_i . We prove this exact statement. We also obtain fixed-oracle exact partners, finite-join intersection formulas, and one jump bound for a whole countable family.

C6: an exact partner of a prescribed c.e. set. For a specific many-one complete c.e. set X , the plan sketches a description $D \approx X$ below $\emptyset''$ with only computable common lower bounds with X . It then asks whether one can additionally require $D' \leq_T \emptyset''$. We prove more: a countable family of such descriptions can be chosen so that its full join is low over $\emptyset'$, and each member and the join have jump exactly $\emptyset''$. The lower bound comes from a uniform classification of the coarse class of X , not from an informal assessment of the construction's complexity.

The other component of C6, concerning exact partners of prescribed Δ_2^0 1-generic sets, is not settled here. Nor are C2 (effective-dense representatives), C7 (sparse 1-generic minimal pairs below $\emptyset'$), or C8 (the uniform least-representative characterization).

1.2 What is inherited and what is proved here

Ingredient	Status and use in this article
Dyadic density coding	Classical; the coarse-description criterion is the relativization of Jockusch–Schupp’s Theorem 2.19 [1]. Reproved in Section 2.
C1 has a negative answer	Already recorded as retired in the plan. The published common-information results of Hirschfeldt–Jockusch–Kuyper–Schupp explain that status [2, 5]. Not a novelty claim here.
Guarded c.e. set and reservoirs	Present in the round-one synthesis [6]. Their definitions and basic proofs are retained explicitly.
Every countable ideal is a core	Proposed in C4 with a sketch. A full proof and strengthened witnesses appear in Sections 5–6.
Prescribed-oracle and joint jump control	The full countable-array theorem, with overlapping finite joins, is the main extension developed here.
Classification of the guarded set	The concrete equivalence $X \equiv_{uc} \mathcal{R}(\text{TOT})$ is proved by two uniform algorithms in Section 3.
Optimal first jump for C6	Derived from the classification and the array theorem in Section 7.

The classical exact-pair problem and refined versions for particular ideals already have a substantial literature; for example, Barmpalias and Downey study exact pairs for the ideal of K -trivial sequences [4]. The present claim is not the invention of exact pairs. It is their simultaneous localization in a single density-zero equality class, with an explicit presentation-dependent bound on the jump of a countable join and an optimal bound for a concrete prescribed target.

1.3 Proof architecture

There are two independent strands. First, we classify the guarded diagonal set by extracting a limit approximation to totality from any coarse description, and constructing a coarse description from any dominating function. Second, we develop a countable product of reservoirs. At each finite stage the protected part is computable in a local oracle B_k , but the effective construction may use a larger oracle H' . A negative disagreement decision then computes a common total output from B_k , not from the full presentation oracle H .

The distinction is essential. An H' -effective construction can yield a very small common lower cone precisely because the decoding argument at a fixed stage uses only local information. Conversely, controlling each D'_i separately would not control $(\bigoplus_i D'_i)'$. We explicitly decide jump computations for the complete joined oracle.

1.4 [write] Notation against the research plan and the other round-2 reports

This article's objects agree with those of the research plan and the synthesis [5, 6]: Core, Comp, the coarse spectra, the columns C_i with $c(i, t) = 2^i(2t + 1) - 1$, the column index $\text{col}(n) = v_2(n + 1)$ (the Lean library's `col`), the dyadic code $\mathcal{R}$ (the plan's `R`, Lean `Rc`), and the guarded set X of (3.1), which is the synthesis's Definition "Guarded diagonal set" and the set of the plan's (K5) and C6 (not the column-only variant of round-1 report 05). No symbol was renamed and no normalization changed when this report was written. The readings most likely to mislead:

Symbol	Meaning here	Do not read it as
$\mathcal{L}(B)$	the lower Turing cone $\{A : A \leq_T B\}$	anything other than $\text{Low}(B)$ of the plan, the synthesis and report 11; only the typeface differs
J, I_n	the block code on $I_n = [2^n - 1, 2^{n+1} - 1]$, so $J(A)(t)$ is the plan's $J(A)(t + 1)$	the plan's J and the Lean <code>blockCode</code> , whose blocks are $[2^n, 2^{n+1})$
F (Section 6)	the robust column profile $F(c(i, t)) = J(A_i)(t)$ of (6.1)	literally the plan's C4 set, which uses the plan's J ; or report 11's Z
F (Sections 4–5)	any target with a layered presentation	the profile only
X	the guarded diagonal set (3.1)	an arbitrary many-one complete c.e. set; or report 11's participant oracles
Z	$\bigoplus_i D_i$, the join of the constructed descriptions	the ideal code, which is F here (and Z in report 11)
H	the presentation oracle; $H = X$ in Section 7	report 11's full join H
P_k, Q_k	protected regions and their complements	a presentation oracle (report 11's P)
$\bigoplus_i D_i$	the dyadic join $Z(c(i, t)) = D_i(t)$; finite joins interleave periodically	a periodic interleaving of infinitely many sets
"high spectrum"	the set (3.5), with no bound $\mathbf{d} \leq \mathbf{0}'$	the high degrees below $\mathbf{0}'$

Report 12 uses X_A for ideal codes different from both F here and report 11's Z . Statements of the round-2 reports should be compared through their definitions, not through their letters.

1.5 [write] Provenance of this report

The report is built from one manuscript: batch 37 manuscript 09, the archive `Exact_Ideals_and_Optimal_Jumps` (inner directory `exact_ideals_coarse/`, main file `article.tex`, 26-page PDF), which arrived in commit `37e61c1fd`. Its pin is the commit printed above, `6c9175a54bcaad3bfc2d416257d2b7d3dff2c1e0` (a batch-36 reciprocal-notes commit), and it was placed as report 13 in commit `0e53d1034`. Between the pin and the placement, `Computability/TuringDegrees/` changed only by the addition of the round-2 reports 11–14; the plan and synthesis blobs quoted in Appendix B are those of the current tree. What the manuscript could not know is the rest of its round: reports 11 and 12 also prove C4, and report 14 takes up C2.

When the report was written, every label received the prefix `cr13:`, and the note on the title page, these four subsections, and [write] notes after Corollary 6.3, in Section 9.3 and in Appendix B were added. No statement of the manuscript was changed. Every numbered statement, equation and section keeps its number in the delivered PDF; that PDF is not shipped, and `article.pdf` is a build of this text.

The delivered files are shipped under new paths: `checks/finite_checks.py` is `code/finite_checks.py`, `checks/results.json` is `data/results.json`, `audit/validation.json` is `data/validation.json`, `audit/PROOF_AUDIT.md` and `audit/SOURCES.md` are `PROOF_AUDIT.md` and `SOURCES.md`, and `build.sh` is `code/build.sh`. Line 8 of `SOURCES.md` records an “initial discovery tree” `1a1396d4d3a2ac6812692df520517d86ba3a4785`, and line 10 calls it “a TREE object, not a commit”. That is false: `1a1396d4d` is a commit (the batch-36 placement commit, whose tree is `c3a58d9d35...`). The commit tree `246be5549...` recorded on line 7 is correct for the pin. The file is shipped unchanged.

1.6 [write] Formal status and the Lean library

No statement of this report is formalized, in Lean or in Rocq; that it sits in the research programme of the Lean/Rocq project `Computability/TuringDegrees` gives it no formal status. The proofs use no published theorem as a black box, so the report relies on none of the eight statements that the project's Lean library `CoarseDegrees` (`Computability/TuringDegrees/Lean/CoarseDegrees`) admits. The library touches the report as follows.

- *Proved in Lean, with nothing admitted, and reproved here.* The counts and density facts of Lemma 2.5: the tail count is `card_tail` (`Columns.lean`, line 74), finitely many errors per column give density zero by `densityZero_of_bounded_columns` (line 85), and restriction to one column is `errCnt_div_tendsto` (`Majority.lean`, line 166). Proposition 2.6 is `description_iff_limit` (`Majority.lean`, line 297) and `spectrum_Rc` (`Spectrum.lean`, line 137) in limit form; the library has no jump operator, so the form “ $A \leq_T B$ ” and the limit lemma, Lemma 2.1, are not stated. The sparse coding of Lemma 2.3 is `exists_description_of_degree` (`BlockChar.lean`, line 42), which implants on $\{2^k - 1\}$.
- *Same argument, different code.* Lemma 2.7 is the majority argument of `blockCode_decode` (`Block.lean`, line 254), but for the shifted code J above; that statement is not in Lean.
- *Not formalized.* Everything else: the guarded set and Proposition 3.1 (the library has no guarded set), Lemmas 3.2–3.4, Theorem 3.5, the reservoir lemmas of Section 4, Theorem 5.1, the C4 results

of Section 6, Theorem 7.1, Corollary 7.2 and the examples. The reservoir constructions need a use-bounded model of Turing functionals, which the library’s README records as unavailable. Section 9.2 is a plan: no Lean or Rocq code was delivered or is shipped. The ledger cited as [7] tracks the separate TuringDegrees library, not CoarseDegrees.

1.7 [write] Programme targets addressed, and what stays open

Line numbers refer to Computability/TuringDegrees/Research/CoarseDegrees/research-plan/turing_degrees_unified.tex (“plan”) and .../research-synthesis/Turing_Degrees_Synthesis.tex (“synthesis”), identical at the pin and in the current tree.

- C6 (plan:603–604; plan:1647 names “the $\emptyset''$ bound in C6” among the quickest gains). For the prescribed guarded set, the plan proposes a partner $D \leq_T \emptyset''$ with $\mathcal{L}(X) \cap \mathcal{L}(D)$ computable and leaves “whether D can have $D' \leq_T \emptyset'''$ ” as a residual question. Theorem 7.1 gives both, with $D'_i \equiv_T \emptyset''$ optimal by Theorem 3.5. The plan’s second residual question, what replaces $\emptyset''$ for a general approximable X , is answered only for targets with a layered presentation (Theorem 5.1: the bound H') and stays open in general (Question 10.3). The exploratory half, a prescribed Δ_2^0 1-generic, stays open (Question 10.1).
- C4 (plan:585–598). Theorem 6.2 and Corollary 6.3 prove the plan’s core formula, for a code that differs from the plan’s by the block shift noted above. The upper inclusion comes from the exactness clause of Theorem 5.1, not from the robust radius (HJKS Theorem 3.7) used by the plan’s sketch and by report 11; this route uses no published black box. Of the tasks at plan:598: the sketch’s conclusion is checked by a different route; this manuscript does not cite HJS 2021, which reports 11 and 12 identify as a close published antecedent of the column code; for which (A_k) the class has a least degree is shown to depend on the presentation, not the ideal (Example 8.2), and stays open in general (Question 10.4); the uniform core is not addressed. The quantitative exact-pair consequence proposed at plan:598 is obtained for a pair and for a countable family inside one density-zero class (Corollaries 5.2 and 6.4), not for the perfect family proposed there (Section 8.3).
- *Classification* (synthesis:973). The ideal half is answered (Corollary 6.3; independently in reports 11 and 12). For spectra, Theorem 3.5 identifies the spectrum of the guarded set as the jump-cone $\{\mathbf{d} : \mathbf{0}'' \leq \mathbf{d}'\}$, a shape the synthesis already realizes; the general spectra question stays open.
- *Effectivity* (synthesis:972). For the guarded set, an exact partner is now bounded by $\emptyset''$ with jump $\emptyset''$, sharpening the synthesis’s “cannot be Δ_2^0 ”. The Δ_2^0 generic case, arithmetical bounds for the perfect families, and witnesses below $\mathbf{0}'$ for the budget theorem stay open.
- *Not addressed*: minimal elements (synthesis:970) and C5 (plan:600), Question 10.5; C2 (plan:563, synthesis:969), Section 8.4 and Question 10.8, the subject of report 14; C7 (plan:606), Question 10.2; C8 (plan:609, synthesis:971).

2 Preliminaries: density, information cores, and coding

2.1 Computability conventions

All sets are subsets of $\mathbb{N} = \{0, 1, 2, \dots\}$, identified with their characteristic functions. Fix acceptable enumerations (φ_e) of partial computable numerical functions and (Φ_e) of oracle functionals. A

finite-oracle computation is declared convergent only when every oracle query is answered by the finite information supplied. Convergence therefore persists under extension. Numerical outputs are allowed in intermediate arguments; claims about sets concern total binary-valued outputs.

Write $A \leq_T B$ for Turing reducibility, $A \equiv_T B$ for equivalence, and A' for the Turing jump. Set

$$\mathcal{L}(B) = \{A \subseteq \mathbb{N} : A \leq_T B\}, \quad \text{Comp} = \mathcal{L}(\emptyset).$$

We freely use the elementary facts that H' decides $\Sigma_1^0(H)$ questions, that $H \leq_T H'$, and that the jump is monotone and strictly increasing on degrees. A finite join uses a fixed computable interleaving. Countable joins will use the dyadic pairing introduced below.

A *Turing ideal*, viewed here as a family of sets, is a nonempty family closed downward under $\leq_T$ and under finite joins. This is equivalent to an ideal of degrees. A countable ideal of degrees contains only countably many sets: below any one oracle there are only countably many functionals. We work classically; no effective enumeration of a countable ideal is assumed.

Lemma 2.1 (Relativized limit lemma). *For sets A, B , $A \leq_T B'$ if and only if there is a total B -computable binary function $a(n, s)$ such that $\lim_s a(n, s) = A(n)$ for every n . The translations are uniform in reduction indices, on oracles satisfying the relevant hypotheses.*

Proof. For the forward implication, approximate B' by its usual B -computable enumeration. On input (n, s) simulate the given B' -oracle program for s steps using the stage- s approximation; output its binary result if one appears, and 0 otherwise. The actual convergent computation asks finitely many questions about B' . Once their answers have stabilized and the simulation time is large enough, this procedure gives the correct result.

Conversely, B' decides, for given n, s , whether

$$(\exists t > s) a(n, t) \neq a(n, s).$$

Search for an s for which the answer is negative and output $a(n, s)$. Pointwise convergence makes this search terminate. Both procedures are program transformations; they do not require a modulus of convergence as extra input. This is the usual limit lemma, relativized [1, Lemma 2.18]. $\square$

2.2 Coarse descriptions and cores

For $E \subseteq \mathbb{N}$ and $N \geq 1$, let

$$\rho_N(E) = \frac{|E \cap [0, N)|}{N}, \quad \bar{\rho}(E) = \limsup_{N \rightarrow \infty} \rho_N(E).$$

Write $D \approx F$ when $\rho_N(D \triangle F) \rightarrow 0$, and let

$$\text{CD}(F) = \{D \in 2^{\mathbb{N}} : D \approx F\}.$$

The *core* of F is

$$\text{Core}(F) = \bigcap_{D \in \text{CD}(F)} \mathcal{L}(D). \tag{2.1}$$

In the notation of Hirschfeldt–Jockusch–Kuyper–Schupp this is the common-information class F^c [2].

The relation $F \leq_{uc} G$ means that one functional sends every coarse description of G to a coarse description of F . The nonuniform relation $F \leq_{nc} G$ allows the functional to depend on the input description. Equivalence is denoted by $\equiv_{uc}$ or $\equiv_{nc}$. Since density-zero symmetric difference is transitive,

$$D \approx F \implies CD(D) = CD(F) \implies D \equiv_{uc} F. \quad (2.2)$$

Lemma 2.2 (Necessary form of a core). *For every F , $\text{Core}(F)$ is a countable Turing ideal contained in $\mathcal{L}(F)$.*

Proof. Every D computes all computable sets. Downward closure and closure under joins hold separately inside each $\mathcal{L}(D)$, hence in their intersection. Since $F \in CD(F)$, the intersection is contained in $\mathcal{L}(F)$, which is countable. $\square$

Lemma 2.3 (Sparse coding and spectra). *The Turing-degree spectra of $CD(F)$, $[F]_{uc}$, and $[F]_{nc}$ are identical. They are upward closed and equal to*

$$\{\deg_T(B) : B \text{ computes a member of } CD(F)\}.$$

Proof. Suppose $D \approx F$ and $D \leq_T B$. On the computable density-zero set $S = \{2^n : n \in \mathbb{N}\}$ replace $D(2^n)$ by $B(n)$, leaving all other bits unchanged. The result E satisfies $E \approx F$, $E \leq_T B$, and $B \leq_T E$ by reading the marked positions. Thus every oracle computing a description has a description of exactly its own degree.

Members of $CD(F)$ are uniformly coarsely equivalent to F by (2.2). Conversely, if $Y \equiv_{nc} F$, apply $F \leq_{nc} Y$ to the exact description Y itself. This gives a Y -computable description of F , and sparse coding produces one of degree $\deg_T(Y)$. The uniform class lies between the two inclusions. These arguments are also the spectrum argument in the repository synthesis [6]. $\square$

Lemma 2.4 (A necessary condition for a least degree). *If the common spectrum in Lemma 2.3 has a least degree $\mathbf{a}$, then $\text{Core}(F)$ is the principal ideal $\{B : \deg_T(B) \leq \mathbf{a}\}$. In particular, a nonprincipal core precludes a least degree.*

Proof. Choose $D \approx F$ of degree $\mathbf{a}$. Every description computes D , so $\mathcal{L}(D) \subseteq \text{Core}(F)$. The reverse inclusion follows by including D in the intersection (2.1). $\square$

The converse is false; a principal core need not give a least representative. Section 8 gives explicit presentations exhibiting this distinction.

2.3 Dyadic columns and the limit code

Define

$$c(i, t) = 2^i(2t + 1) - 1, \quad C_i = \{c(i, t) : t \in \mathbb{N}\}, \quad T_k = \bigcup_{i \geq k} C_i. \quad (2.3)$$

The map $c : \mathbb{N}^2 \rightarrow \mathbb{N}$ is a computable bijection, and $\text{col}(n) = v_2(n + 1)$ is its first inverse coordinate.

Lemma 2.5 (Column estimates). *For $N \geq 1$,*

$$|C_i \cap [0, N]| = \left\lfloor \frac{N}{2^i} \right\rfloor - \left\lfloor \frac{N}{2^{i+1}} \right\rfloor, \quad |T_k \cap [0, N]| = \left\lfloor \frac{N}{2^k} \right\rfloor.$$

Thus $\rho(C_i) = 2^{-i-1}$ and $\rho_N(T_k) \leq 2^{-k}$. If $E \cap C_i$ is finite for every i , then $\rho_N(E) \rightarrow 0$. If $\rho_N(E) \rightarrow 0$, then $\{t : c(i, t) \in E\}$ has density zero for every fixed i .

Proof. The first identities count divisibility of the integers $1, \dots, N$. For finite errors per column, choose M containing all errors in $C_0, \dots, C_{k-1}$. Then

$$\rho_N(E) \leq M/N + 2^{-k}. \quad (2.4)$$

First let $N \rightarrow \infty$, then $k \rightarrow \infty$. For restriction to one column, the first s positions of C_i lie below $2^{i+1}s$, so their error proportion is at most $2^{i+1}\rho_{2^{i+1}s}(E)$. $\square$

The tail estimate, rather than countability of the columns, justifies the limit. An arbitrary countable union of finite sets need not have density zero.

Put $\mathcal{R}(A)(c(i, t)) = A(i)$. This code stores one bit on each positive-density column. It satisfies $\mathcal{R}(A) \equiv_T A$.

Proposition 2.6 (Description criterion for the dyadic code). *An oracle B computes a coarse description of $\mathcal{R}(A)$ if and only if $A \leq_T B'$. From a description D there is a uniform D -computable limit approximation to A , and hence a uniform computation of A from D' .*

Proof. Given $D \approx \mathcal{R}(A)$, let $a(i, s)$ be the majority of D on the first $s + 1$ elements of C_i , breaking ties in favor of 0. By Lemma 2.5, the relative error on this column tends to zero. Thus $a(i, s) \rightarrow A(i)$, and Lemma 2.1 applies.

Conversely, given a total B -computable approximation $a(i, s) \rightarrow A(i)$, define $D(n) = a(\text{col}(n), n)$. In each fixed column all errors occur before stabilization. Lemma 2.5 implies $D \approx \mathcal{R}(A)$. This is the relativized classical argument [1, Theorem 2.19]. $\square$

2.4 A robust code and countable repetition

Let

$$I_n = [2^n - 1, 2^{n+1} - 1), \quad J(A)(t) = A(n) \quad (t \in I_n).$$

The length of I_n is 2^n . This code differs from $\mathcal{R}$ in that it stores successive bits on increasingly long consecutive intervals.

Lemma 2.7 (Robust block decoding). *If $D \approx J(A)$, then $A \leq_T D$. Also $J(A) \leq_T A$.*

Proof. Compute the majority of D on each I_n . If the majority is wrong, that block contains at least 2^{n-1} errors for $n \geq 1$. At its right endpoint the cumulative error density is then at least

$$\frac{2^{n-1}}{2^{n+1} - 1} > \frac{1}{4}.$$

Infinitely many wrong majorities would contradict density-zero error. The majority sequence is consequently a finite variant of A . Hard-code its finitely many corrections to compute A from D . The forward coding is plainly computable from A . $\square$

The finite correction is nonuniform: this lemma does not provide one exact decoder working on every description. No such stronger conclusion is used.

For a set F , define its countable repetition by

$$\text{Rep}(F)(c(i, t)) = F(t).$$

For a sequence (D_i) write $Z = \bigoplus_i D_i$ for $Z(c(i, t)) = D_i(t)$.

Lemma 2.8 (Joining descriptions). *If $D_i \approx F$ for every i , then $\bigoplus_i D_i \approx \text{Rep}(F)$ and $\text{Rep}(F) \equiv_{\text{uc}} F$. Every nonempty finite join of the D_i is also uniformly coarsely equivalent to F . Moreover, the degree of the full join is realized by a member of $\text{CD}(F)$.*

Proof. For the countable join, fix k . Errors in each of the first k columns have density zero; the remaining errors lie in T_k . Lemma 2.5 bounds their upper density by 2^{-k} , and k is arbitrary. No uniform rate of convergence for (D_i) is needed.

Repeating an input description on all columns gives one reduction $\text{Rep}(F) \leq_{\text{uc}} F$; restriction to C_0 gives $F \leq_{\text{uc}} \text{Rep}(F)$. Both statements use the same density estimates. Finite interleaving works by finite additivity and restriction to a positive-density residue class. Finally, the full join computes $D_0 \approx F$, so Lemma 2.3 supplies a description of its exact degree. $\square$

3 The guarded diagonal set has exactly the high spectrum

3.1 The prescribed c.e. target

We use the precise guarded set from the repository synthesis, not an arbitrary complete c.e. set. For $n \in C_e$, define

$$X(n) = 1 \iff [\forall m \leq n \varphi_e(m) \downarrow \in \{0, 1\}] \text{ and } \varphi_e(n) = 0. \quad (3.1)$$

The condition has a finite positive witness, so X is c.e.

Proposition 3.1 (Basic properties of the guarded set). *The set X is many-one complete c.e., hence $X \equiv_{\text{T}} \emptyset'$. For each fixed e , $X \cap C_e$ is computable, although the indices need not be uniformly computable in e . No computable set is a coarse description of X .*

Proof. If φ_e is total and binary-valued, then $X(n) = 1 - \varphi_e(n)$ for $n \in C_e$. Otherwise choose the least input m on which it is undefined or nonbinary. For every $n \geq m$ the guard fails. Hence $X \cap C_e$ is finite in the latter case. This is a case distinction proving the existence of a computable section, not an effective test for totality.

By the parameter theorem, from p obtain an index $h(p)$ for a program that waits for $\varphi_p(p)$ to halt and then outputs 0, independently of its input. This program is either total zero or nowhere defined. Thus

$$p \in \emptyset' \iff c(h(p), 0) \in X.$$

This proves many-one completeness. Finally, if C is computable, choose e with $\varphi_e = \chi_C$. The sets X and C disagree everywhere on C_e , a set of positive density. Hence $C \not\approx X$. $\square$

The use of a guard is not cosmetic. Without it, a partial program can leave an arbitrary c.e. pattern inside a positive-density column, destroying the computable-section property needed later.

3.2 Recovering totality in the limit

Let $\text{TOT} = \{e : \varphi_e \text{ is total}\}$. We use the standard completeness fact $\text{TOT} \equiv_{\text{T}} \emptyset''$. To recall why, totality has the form $\forall n \exists s$ with a computable matrix. Any such predicate can be reduced to totality by a program which, on input n , searches for witnesses for all requirements up to n . Thus TOT is Π_2^0 -complete and has degree $\emptyset''$.

Lemma 3.2 (A uniform totality approximation). *There is a computable function f such that, for every e , $X \cap C_{f(e)}$ is all of $C_{f(e)}$ when $e \in \text{TOT}$ and is finite otherwise. Consequently every $D \approx X$ uniformly computes a total binary approximation $a_D(e, s) \rightarrow \text{TOT}(e)$.*

Proof. Let $f(e)$ be an index for the program which, on input n , waits until $\varphi_e(m)$ has halted for every $m \leq n$, then outputs 0. If φ_e is total, this new program is total zero. If not, it fails from the least divergent input onward. Apply (3.1) to the column with index $f(e)$.

Let $a_D(e, s)$ be the majority of D on the first $s + 1$ points of $C_{f(e)}$. In the total case the relative density of 1's tends to one; in the nontotal case it tends to zero. Restricting density-zero errors to that column is legitimate by Lemma 2.5. The algorithm makes finitely many queries to D and uses no totality oracle. $\square$

In particular, $\emptyset'' \leq_T D'$ for every $D \approx X$. This is a lower bound on every description, not just on descriptions produced by a particular method.

3.3 From a totality approximation to a dominating function

Lemma 3.3 (Uniform domination from limit totality). *Suppose an oracle B computes a total binary function $a(e, s)$ converging pointwise to $\text{TOT}(e)$. Then B uniformly computes a total numerical function g which eventually strictly dominates every total computable numerical function.*

Proof. On input n , search for a stage $s \geq n$ with the following property:

$$\text{for every } e \leq n \text{ with } a(e, s) = 1, \varphi_e(n) \text{ halts within } s \text{ steps.} \quad (3.2)$$

For this fixed finite collection of indices, the approximation eventually stabilizes. The indices then marked 1 are precisely the total ones in that collection, and all their computations at n halt. A sufficiently large stage therefore satisfies (3.2). At the first successful stage, put

$$g(n) = 1 + \max(\{n\} \cup \{\varphi_e(n) : e \leq n \text{ and } a(e, s) = 1\}). \quad (3.3)$$

All values in the maximum have been found by the bounded simulations, so g is total and B -computable.

Fix a total φ_e . Choose s_0 so that $a(e, s) = 1$ for all $s \geq s_0$. For every $n \geq \max(e, s_0)$, the selected stage satisfies $s \geq n \geq s_0$, so the maximum includes $\varphi_e(n)$. Hence $g(n) > \varphi_e(n)$ for all such n . Notice that the algorithm never needs to know s_0 . $\square$

Lemma 3.4 (A dominating oracle computes a description of X). *There is one procedure which, from any total function g eventually dominating every total computable function, computes a coarse description of X .*

Proof. For input $n \in C_e$, simulate every $\varphi_e(m)$ with $m \leq n$ for $g(n)$ steps each. Output 1 exactly when all these simulations halt with binary values and the value at n is 0; otherwise output 0. This is a total computation from g .

If φ_e is not total binary, let m be its least bad input. For $n \geq m$, the procedure outputs 0, exactly as X does. If φ_e is total binary, define the total computable runtime function

$$r_e(n) = \max\{\text{halting time of } \varphi_e(m) : m \leq n\}.$$

Eventually $g(n) \geq r_e(n)$, and then the procedure agrees with the guard definition at n . Thus there are finitely many errors in each column. Lemma 2.5 proves density-zero total error. $\square$

Theorem 3.5 (Uniform classification of the guarded set). *For the set X in (3.1),*

$$X \equiv_{\text{uc}} \mathcal{R}(\text{TOT}). \quad (3.4)$$

Its description spectrum, uniform coarse-class spectrum, and nonuniform coarse-class spectrum are all

$$\{\mathbf{d} : \mathbf{0}'' \leq \mathbf{d}'\}. \quad (3.5)$$

Here the inequality in (3.5) is not accompanied by a restriction $\mathbf{d} \leq \mathbf{0}'$.

Proof. Given $D \approx X$, use Lemma 3.2 to obtain $a_D(e, s) \rightarrow \text{TOT}(e)$ and output $a_D(\text{col}(n), n)$ on input n . The second half of Proposition 2.6 shows that this is a description of $\mathcal{R}(\text{TOT})$. The procedure is uniform in D .

Given $E \approx \mathcal{R}(\text{TOT})$, column majorities give an E -computable approximation to TOT . Apply Lemma 3.3, then Lemma 3.4, to produce a description of X . These are fixed oracle procedures and hence establish the opposite uniform reduction. None of them queries E' as an oracle.

Finally, Proposition 2.6 says that B computes a description of $\mathcal{R}(\text{TOT})$ exactly when $\text{TOT} \leq_T B'$, equivalently $\emptyset'' \leq_T B'$. Combine the uniform equivalence with Lemma 2.3. $\square$

Remark 3.6. The phrase “high spectrum” is convenient, but terminology about high degrees is often used within the interval below $\mathbf{0}'$. Formula (3.5), rather than that informal convention, is the assertion used here. In particular, the later exact partners are not below $\emptyset'$ even though their jumps equal $\emptyset''$.

4 Layered reservoirs and the finite-extension lemmas

4.1 Two oracles with different jobs

Fix a presentation oracle H , a target $F \leq_T H$, and increasing sets

$$P_0 \subseteq P_1 \subseteq \cdots \subseteq \mathbb{N}.$$

Assume that membership in P_k is uniformly computable from (H, k) , and set $Q_k = \mathbb{N} \setminus P_k$. We require

$$Q_k \text{ infinite for every } k, \quad \lim_{k \rightarrow \infty} \bar{\rho}(Q_k) = 0. \quad (4.1)$$

Let $B_0 \leq_T B_1 \leq_T \cdots \leq_T H$ and define

$$\mathcal{I} = \bigcup_k \mathcal{L}(B_k). \quad (4.2)$$

For each fixed k , assume that P_k and $F \cap P_k$ are B_k -computable. The existence of these local computations is sufficient; no uniform list of their indices is part of the hypothesis. Finally assume

$$\mathcal{I} \subseteq \text{Core}(F). \quad (4.3)$$

The task of a particular application is to verify (4.3). It is not asserted for every reservoir presentation. For computable B_k it is automatic; for arbitrary ideals it will follow from robust block decoding.

The oracle H has enough information to decide finite admissibility uniformly. The smaller oracle B_k is used only after a single stage and its finite data have been fixed, to compute a common output. Conflating these two uses would replace the desired ideal by an unnecessarily large lower cone.

4.2 Finite array conditions

Definition 4.1 (Condition and reservoir). A condition $p = (k, \sigma)$ consists of a level k and an array $(\sigma_i)_{i \in \mathbb{N}}$ of finite binary strings with only finitely many nonempty entries. Its reservoir $[p]$ is the set of all arrays $(Y_i)_{i \in \mathbb{N}}$ such that, for every i ,

$$Y_i \geq \sigma_i, \quad n \geq |\sigma_i| \ \& \ n \in P_k \implies Y_i(n) = F(n). \quad (4.4)$$

A finite array τ is admissible over p if it extends every σ_i , has finite support, and satisfies

$$n \in [|\sigma_i|, |\tau_i|) \cap P_k \implies \tau_i(n) = F(n).$$

Finite support here means finitely many nonempty strings, not finitely many nonzero bits in the eventual infinite arrays.

Every reservoir is nonempty: extend each stem by F on protected positions and by 0 elsewhere. An admissible finite array has the same property. Given H , admissibility is decidable uniformly in its finite arguments. At a fixed level it is also decidable from B_k , with a suitable finite choice of local indices.

The joined finite oracle Z_τ answers a query $c(i, t)$ precisely when $t < |\tau_i|$, with answer $\tau_i(t)$. For a finite set S of indices, $W_S(\tau)$ is the finite oracle obtained by interleaving the strings with indices in S , in increasing index order. Missing bits remain unanswered. Let $W_\emptyset$ denote the computable empty oracle.

Lemma 4.2 (Preservation and finite witnesses). *If τ is admissible over $p = (k, \sigma)$ and $\ell \geq k$, then*

$$[\ell, \tau] \subseteq [p].$$

Every convergent computation on an array in $[p]$ has an admissible finite-array witness over p . Every convergent computation witnessed by such an array persists in all reservoirs extending that witness.

Proof. The newly assigned portions of the stems respect P_k by admissibility. Beyond the new stems, agreement on P_ℓ implies agreement on P_k because the protected sets are increasing. Existing stem bits need not agree with F and are never changed.

A convergent oracle computation asks only finitely many questions. For every coordinate that appears, take a long enough prefix of the actual array member, and also retain the finitely many old stems. This gives the witness. Persistence is the finite-use property of oracle computation. $\square$

This lemma also proves persistence of a negative statement saying that no admissible witness exists. Such a statement is made about the entire current reservoir, and every later reservoir is a subset of it.

4.3 One side fixed: exactness against the presentation oracle

Lemma 4.3 (Fixed-oracle disagreement lemma). *Fix $p = (k, \sigma)$ and indices e, j . Consider the question*

$$\exists \tau, n \quad \tau \text{ admissible over } p \ \& \ \Phi_e^{Z_\tau}(n) \downarrow \neq \Phi_j^H(n) \downarrow. \quad (4.5)$$

It is a $\Sigma_1^0(H)$ question. If the answer is yes, a witness permanently forces disagreement. If the answer is no and an array in $[p]$ has join Z with

$$\Phi_e^Z = \Phi_j^H = A$$

as total functions, then $A \leq_T B_k$.

Proof. Finite admissibility is H -decidable, and both convergences in (4.5) can be searched for using H . This proves the complexity assertion. A positive witness persists by Lemma 4.2.

For the negative case, fix indices witnessing the B_k -computability of the protected set and protected values. On input n , enumerate all finite arrays admissible over p , dovetail the computations $\Phi_e^{Z_\tau}(n)$, and return the first convergent value. This is a B_k -oracle algorithm with finitely many hard-coded parameters. It terminates because the actual computation $\Phi_e^Z(n)$ has a finite admissible witness. Every returned value equals $\Phi_j^H(n) = A(n)$; otherwise it would be a witness to (4.5). The algorithm never calls H or Φ_j^H to check its answer. $\square$

A separate totality test is unnecessary. Totality is a hypothesis only in the verification of a common lower bound. This avoids introducing a second jump over H .

4.4 Two finite joins with shared coordinates

The usual product disagreement lemma treats independent sides. Finite joins can overlap, so it must be used with care. If a coordinate occurs on both sides, it has one value in the underlying array; choosing incompatible values on the two sides would not witness any possible disagreement.

Lemma 4.4 (Overlap-preserving disagreement lemma). *Fix nonempty finite sets $S, T \subseteq \mathbb{N}$, put $U = S \cap T$, and fix $p = (k, \sigma)$ and indices e, j . Ask whether*

$$\exists \tau, n \quad \tau \text{ admissible over } p \quad \& \quad \Phi_e^{W_S(\tau)}(n) \downarrow \neq \Phi_j^{W_T(\tau)}(n) \downarrow. \quad (4.6)$$

This is $\Sigma_1^0(H)$. If no witness exists and an actual array in $[p]$ satisfies $\Phi_e^{W_S} = \Phi_j^{W_T} = A$ total, then

$$A \leq_T B_k \oplus W_U. \quad (4.7)$$

Proof. The complexity and positive-witness assertions are as before. In the negative case, use $B_k \oplus W_U$ to search finite admissible extensions on the coordinates in S , requiring that all assigned bits in the shared coordinates U agree with the actual oracle W_U . Dovetail Φ_e on these finite S -oracles and return the first convergent value. Old stems on all other coordinates are retained. Finite agreement with W_U and admissibility are decidable in the stated oracle.

An actual finite witness for $\Phi_e^{W_S}(n)$ is among the candidates, so the search halts. Suppose it returns $v \neq A(n)$. Take an actual finite witness for $\Phi_j^{W_T}(n) = A(n)$. Its strings on U are compatible with those of the candidate, because both agree with the actual shared oracle. Amalgamate the two finite arrays: on $S \setminus T$ use the candidate strings, on $T \setminus S$ use the actual strings, on U use the longer of the compatible strings, and elsewhere retain the old stems. The amalgam is admissible over p . Both convergences persist, contradicting the negative answer to (4.6). Thus every returned value is correct, proving (4.7). $\square$

The conclusion includes the genuinely independent case $U = \emptyset$, when the common output is B_k -computable. In the overlapping case it says that the only additional common information is already available on the shared coordinates.

4.5 Avoidance and jump decisions

Lemma 4.5 (Avoiding a fixed oracle). *Given $p = (k, \sigma)$ and i, e , an H' -computable extension ensures that the eventual i th coordinate differs from Φ_e^H as a total binary function.*

Proof. Using H , find $n \geq |\sigma_i|$ with $n \in Q_k$. Such an n exists by (4.1). Use H' to decide whether $\Phi_e^H(n)$ halts. If it diverges, the requirement is already satisfied. If it converges to v , extend σ_i through n and set the bit at n to 1 when $v = 0$, and to 0 otherwise. Fill all intermediate protected positions with F and all other new positions arbitrarily, say with 0. The position n is unprotected, so the extension is admissible, and its bit differs from v even if v is nonbinary. $\square$

Lemma 4.6 (Deciding the full joined jump). *For each $p = (k, \sigma)$ and e , the question*

$$\exists \tau \quad \tau \text{ admissible over } p \quad \& \quad \Phi_e^{Z_\tau \oplus H}(e) \downarrow \quad (4.8)$$

is $\Sigma_1^0(H)$. A positive answer can be forced by a finite extension; a negative answer implies divergence for every array in $[p]$.

Proof. Enumerate finite arrays, check admissibility with H , and simulate using the finite array oracle together with the full fixed oracle H . This gives a positive-witness search relative to H . In the negative case any convergent computation on an array in $[p]$ would, by finite use, produce a witness to (4.8). $\square$

The fixed oracle in the jump test is H , not H' . The construction uses H' to answer these H -c.e. questions. Dependence of earlier finite conditions on H' does not increase their complexity: at each new stage those conditions are finite parameters, which can be inserted into an index for an H -oracle search.

5 The countable-array exact-ideal theorem

For finite $S \subseteq \mathbb{N}$ write $W_S = \bigoplus_{i \in S} D_i$, with $W_\emptyset = \emptyset$. For a finite U define

$$\mathcal{I}[W_U] = \bigcup_k \mathcal{L}(B_k \oplus W_U). \quad (5.1)$$

Thus $\mathcal{I}[W_\emptyset] = \mathcal{I}$. If $U \neq \emptyset$ and every D_i is a description of F , hypothesis (4.3) implies $\mathcal{I}[W_U] = \mathcal{L}(W_U)$.

Theorem 5.1 (Layered exact ideals with joint jump control). *Assume all the hypotheses of Section 4: $F \leq_T H$, the protected sets P_k are uniformly H -computable and increasing, (4.1) holds, $B_0 \leq_T B_1 \leq_T \dots \leq_T H$, each P_k and $F \cap P_k$ is B_k -computable, and $\mathcal{I} = \bigcup_k \mathcal{L}(B_k) \subseteq \text{Core}(F)$.*

There is a sequence $(D_i)_{i \in \mathbb{N}}$, with $Z = \bigoplus_i D_i$, satisfying all of the following:

$$D_i \approx F \quad \text{for every } i; \quad (5.2)$$

$$\mathcal{L}(Z) \cap \mathcal{L}(H) = \mathcal{I}; \quad (5.3)$$

$$D_i \not\leq_T H \quad \text{for every } i; \quad (5.4)$$

$$(Z \oplus H)' \equiv_T H', \quad Z \leq_T H'; \quad (5.5)$$

$$\mathcal{L}(W_S) \cap \mathcal{L}(W_T) = \mathcal{I}[W_{S \cap T}] \quad \text{for all nonempty finite } S, T. \quad (5.6)$$

In particular, the right side of (5.6) is $\mathcal{I}$ when $S \cap T = \emptyset$, and is $\mathcal{L}(W_{S \cap T})$ otherwise. Each nonempty finite join, and also the degree of Z , is represented in the uniform coarse class of F . The degree of Z has a representative in $\text{CD}(F)$ itself.

Proof. **The list of requirements.** Fix a computable enumeration of the following countable collection, with each requirement occurring at least once:

$$\begin{aligned} R_{e,j} : \quad & \Phi_e^Z = \Phi_j^H = A \text{ total} \implies A \in \mathcal{I}; \\ M_{S,T,e,j} : \quad & \Phi_e^{W_S} = \Phi_j^{W_T} = A \text{ total} \implies A \in \mathcal{I}[W_{S \cap T}]; \\ N_{i,e} : \quad & D_i \neq \Phi_e^H \text{ as total binary functions}; \\ J_e : \quad & \text{decide whether } \Phi_e^{Z \oplus H}(e) \text{ halts.} \end{aligned}$$

Finite subsets have computable codes, so the second family is effectively enumerable. We will make only finite extensions, and negative decisions will be preserved by reservoir inclusion. No injury-and-repair argument is needed.

Stage construction. Begin with $p_0 = (0, \emptyset)$. At stage s the current condition is $p_s = (s, \sigma^s)$. Process the s th requirement on the list.

For $R_{e,j}$, use H' to decide (4.5). If positive, search with H for a witness and extend to it. If negative, leave the stems unchanged and retain the negative decision.

For $M_{S,T,e,j}$, do the same with (4.6), using a single finite array for both sides. For $N_{i,e}$, apply Lemma 4.5. For J_e , decide (4.8); extend to a positive witness or retain the negative decision. Record the resulting bit as $b(e)$, using one designated occurrence if the schedule repeats a jump requirement.

After the requirement has been processed, extend the stems in every coordinate $i \leq s$ to length at least $s + 1$. Use F on protected positions and 0 on free positions not already assigned. Then raise the level from s to $s + 1$, without changing the stems. This defines p_{s+1} . Each step is H' -computable, and Lemma 4.2 gives $[p_{s+1}] \subseteq [p_s]$.

Existence of the limit and its complexity. Define

$$D_i = \bigcup_s \sigma_i^s.$$

The strings in a fixed coordinate extend one another. Once $s \geq i$, the length requirement gives $|\sigma_i^{s+1}| \geq s + 1$, so every D_i is total. To compute $D_i(n)$ from H' , simulate stages until its stem has length greater than n . Thus the joined set Z is H' -computable, uniformly in the fixed presentation data. The limit array belongs to every reservoir $[p_s]$: any later assigned bit on an already protected region agrees with F unless it was in an earlier, permanently fixed stem.

Density verification. Fix i, k , and let $\ell = |\sigma_i^k|$. Membership of the limit array in $[p_k]$ yields

$$D_i \triangle F \subseteq [0, \ell) \cup Q_k, \quad \rho_N(D_i \triangle F) \leq \ell/N + \rho_N(Q_k).$$

Taking a limsup in N and then letting $k \rightarrow \infty$ proves (5.2). The cutoff ℓ can depend on both i and k ; a uniform density modulus is neither asserted nor needed.

Exactness against H . Suppose $A \leq_T Z, H$. Choose indices e, j computing its characteristic function. At the stage for $R_{e,j}$, a positive disagreement could not have occurred, because it would persist and contradict equality of the two actual functions. The decision was therefore negative. Lemma 4.3 gives $A \leq_T B_k$ for the level k used at that stage. Hence $A \in \mathcal{I}$.

Conversely, if $A \in \mathcal{I}$, then $A \leq_T H$ by (4.2). By (4.3) and (5.2), every D_i computes A , so Z computes A . This proves (5.3). Lemma 4.5, at the stages for $N_{i,e}$, proves (5.4).

Finite-join intersections. Suppose $A \leq_T W_S, W_T$ for fixed nonempty finite S, T , and select reduction indices e, j . The stage for $M_{S,T,e,j}$ must have a negative answer, again because positive disagreement would persist. Lemma 4.4 puts A below $B_k \oplus W_{S \cap T}$ for that stage's k , proving the forward inclusion in (5.6).

For the reverse inclusion, every B_k is computed by every D_i , by (4.3); since S, T are nonempty, both joins compute B_k . They also both compute the shared join. Thus every set in $\mathcal{I}[W_{S \cap T}]$ lies below both. When $S \cap T \neq \emptyset$, any one shared coordinate computes every B_k , so the union in (5.1) equals $\mathcal{L}(W_{S \cap T})$.

The jump of the full join. The recorded bit $b(e)$ is H' -computable: simulate the finite construction through the designated stage for J_e . If that bit is 1, its convergence witness persists to the limit. If it is 0 and the limit computation nevertheless converges, its finite use supplies an admissible witness over that stage's condition, contradicting the negative answer. Consequently

$$e \in (Z \oplus H)' \iff b(e) = 1.$$

This proves $(Z \oplus H)' \leq_T H'$. The reverse reduction follows from $H \leq_T Z \oplus H$ and monotonicity of the jump. In particular this is a decision procedure for the actual jump, not merely an enumeration of convergences found during the construction.

Finally, Lemma 2.8 proves the coarse-class and exact-degree realization claims. $\square$

5.1 Consequences that must not be inferred from weaker statements

The theorem decides the jump of $Z \oplus H$ directly. It does not use a false distributivity principle for jumps, nor does it infer the complexity of a countable join from the separate complexities of its columns.

The lowness statement in (5.5) is *lowness over H* : after adjoining H , the constructed oracle adds no jump power beyond H' . It does not say that Z is computable from H . Indeed, (5.4) implies $Z \not\leq_T H$. It also does not, in general, say that $Z' \equiv_T H'$; only $Z' \leq_T H'$ follows immediately. Equality will require an independent lower bound in the C6 application.

Corollary 5.2 (Order and meet pattern of finite joins). *Under Theorem 5.1, for nonempty finite S, T ,*

$$W_S \leq_T W_T \iff S \subseteq T.$$

Joins correspond to unions of index sets. If $S \cap T \neq \emptyset$, the greatest common lower degree is $\deg_T(W_{S \cap T})$. If $S \cap T = \emptyset$, the common lower cone is exactly $\mathcal{I}$: it has a greatest degree precisely when $\mathcal{I}$ is principal.

Proof. Inclusion of index sets plainly gives the reduction. Conversely, choose $i \in S \setminus T$ if inclusion fails. Then $D_i \leq_T W_S \leq_T W_T$. Apply (5.6) to $\{i\}$ and T to obtain $D_i \in \mathcal{I} \subseteq \mathcal{L}(H)$, contradicting (5.4). The meet assertions restate the exact lower-cone formula, using the definition of a principal ideal. $\square$

For $\mathcal{I} = \text{Comp}$, adjoining the computable degree as the image of the empty set realizes the finite-subset lattice by these degrees and their ordinary Turing meets. For a nonprincipal ideal, disjoint joins have no meet at all; it would be wrong to call that case a lattice embedding. In both cases all nonempty joins belong to the same nontrivial uniform coarse class whenever F is not coarsely computable.

Corollary 5.3 (A literal description exact against the prescribed target). *Under Theorem 5.1, there is $E \approx F$ such that*

$$E \equiv_T Z, \quad \mathcal{L}(E) \cap \mathcal{L}(H) = \mathcal{I}, \quad (E \oplus H)' \equiv_T H'.$$

If moreover $F \equiv_T H$, then F and E are an exact pair for $\mathcal{I}$ inside the single density-zero equality class $\text{CD}(F)$.

Proof. Overwrite D_0 on the powers of two with the bits of Z , as in Lemma 2.3. Since $D_0 \leq_T Z$, the resulting E has degree exactly that of Z and still describes F . The other statements depend only on degrees. $\square$

6 Every countable Turing ideal is a core: target C4

6.1 The proposed code, with all hypotheses checked

Let $(A_i)_{i \in \mathbb{N}}$ be any sequence of sets. No computability assumption is made on the sequence. Define

$$H = \bigoplus_i A_i, \quad B_k = \bigoplus_{i < k} A_i, \quad F(c(i, t)) = J(A_i)(t), \quad (6.1)$$

with $B_0 = \emptyset$. The associated ideal is

$$\mathcal{I} = \bigcup_k \mathcal{L}(B_k). \quad (6.2)$$

This is the robust column profile proposed in C4 of the research plan [5].

Lemma 6.1 (Local information and robust lower inclusion). *For the profile in (6.1), $F \equiv_T H$ and $\mathcal{I} \subseteq \text{Core}(F)$. With $P_k = \bigcup_{i < k} C_i$, all the hypotheses of Theorem 5.1 hold.*

Proof. Given H , compute A_i uniformly in i , and then compute the block code at any requested position. Thus $F \leq_T H$. Conversely,

$$A_i(n) = F(c(i, 2^n - 1)),$$

so the full joined set H is computable from F .

Let $D \approx F$, and fix i . The column restriction $D_i^*(t) = D(c(i, t))$ is a coarse description of $J(A_i)$ by the final assertion of Lemma 2.5. Lemma 2.7 gives $A_i \leq_T D_i^* \leq_T D$. For each finite k , combine these finitely many reductions to get $B_k \leq_T D$. Since D was arbitrary, (4.3) follows. The reductions may depend on i and D ; only their existence for a fixed finite collection is needed.

The protected sets P_k are uniformly computable, their complements are T_k , and $\rho_N(T_k) \leq 2^{-k}$. Every T_k is infinite. On P_k , computing F requires only $A_0, \dots, A_{k-1}$, so $F \cap P_k$ is B_k -computable. Finally $B_k \leq_T H$ and the B_k increase under Turing reducibility. Thus every hypothesis of the array theorem has been verified. $\square$

Theorem 6.2 (Exact core of a robust column profile). *For every sequence (A_i) and F defined by (6.1),*

$$\text{Core}(F) = \bigcup_k \mathcal{L}(A_0 \oplus \dots \oplus A_{k-1}). \quad (6.3)$$

Furthermore, there are descriptions (D_i) and E of F such that, with $Z = \bigoplus_i D_i$,

$$E \equiv_T Z, \quad (Z \oplus H)' \equiv_T H', \quad \mathcal{L}(F) \cap \mathcal{L}(E) = \mathcal{I},$$

and the entire finite-join intersection pattern of Theorem 5.1 holds.

Proof. The previous lemma gives the inclusion $\mathcal{I} \subseteq \text{Core}(F)$ and allows us to apply Theorem 5.1. If $A \in \text{Core}(F)$, then $A \leq_T F \equiv_T H$ because F is its own description. Also $A \leq_T D_0 \leq_T Z$. The exactness clause $\mathcal{L}(Z) \cap \mathcal{L}(H) = \mathcal{I}$ therefore gives $A \in \mathcal{I}$. This proves (6.3). The additional assertions are the array theorem and Corollary 5.3. $\square$

Corollary 6.3 (Classification of all possible cores). *A family of sets is $\text{Core}(F)$ for some binary set F if and only if it is a countable Turing ideal. Equivalently, the possible degree ideals of common information in a coarse description class are exactly the countable Turing ideals.*

Proof. Necessity is Lemma 2.2. For sufficiency, enumerate a countable ideal by sets $A_0, A_1, \dots$, allowing repetitions. Because the ideal is closed under finite joins and downward reduction, it equals the union of the lower cones of their finite joins. Apply Theorem 6.2. $\square$

The enumeration in this proof is not asserted to be computable. Its full join is the presentation oracle H , and the effective bound is measured relative to that presentation. This qualification is particularly important when every individual A_i is computable but the sequence of their indices is not.

[write] Corollary 6.3 is proved independently in the other C4 reports of the same round. Report 11 (directory `../11`) proves it for the plan’s literal code, with the plan’s robust-radius sketch written out. Report 12 (directory `../12`), which merges three manuscripts, proves it for other codes and adds core-neutral joins and an order embedding of a Turing upper cone into the sets with one prescribed core. The proof here differs from both: its upper inclusion is the exactness clause of Theorem 5.1, not a robust-radius (cone-avoiding compactness) argument. Reports 11 and 12 identify the column-coding architecture as already present in Hirschfeldt–Jockusch–Schupp (2021, Theorem 3.11(2)), which this manuscript does not cite; the “completion” of C4 claimed in Section 1 is a proof of the plan’s proposal, not a new phenomenon. None of these proofs is formalized (Section 1.6).

6.2 Localized exact pairs and absent greatest lower bounds

Corollary 6.4 (Exact pairs within one density-zero class). *Every countable Turing ideal $\mathcal{I}$ is the common lower cone of two sets that differ on a density-zero set. They can be chosen as F, E in Theorem 6.2, with $F \equiv_T H$ and $(E \oplus F)' \equiv_T H'$.*

If $\mathcal{I}$ is nonprincipal, these two degrees are incomparable and have no greatest lower bound. The same is true of any two disjoint nonempty finite joins of the constructed D_i .

Proof. Both F and E describe F , so $E \approx F$, and the exact lower cone is (6.2). A greatest common lower degree would generate that ideal, contrary to nonprincipality. If one degree were below the other, its principal lower cone would be their intersection, giving the same contradiction. The finite-join assertion follows from Corollary 5.2. $\square$

Corollary 6.5 (Strict chains and nonexistence of a least upper bound). *If the profile is chosen so that $B_0 <_T B_1 <_T \dots$, the sequence $(\deg_T(B_k))$ has no least upper bound. It has exact upper bounds F, E inside one density-zero equality class, and neither of those bounds equals any B_k .*

Proof. Every B_k lies below both F and E . If a least upper bound L existed, it would lie below both of them and hence below some B_k by exactness. Since it also bounds B_{k+1} , this contradicts strict increase. A member B_k cannot itself be an upper bound for the strict chain. $\square$

The order-theoretic consequence is classical once an exact pair is available. The extra content here is the placement of the pair in the prescribed coarse description class and the effective bound attached to the explicit profile.

7 Optimal exact partners of the guarded set: target C6

Apply Theorem 5.1 with

$$F = X, \quad H = X \equiv_T \emptyset', \quad P_k = \bigcup_{e < k} C_e, \quad B_k = \emptyset \quad \text{for all } k. \quad (7.1)$$

Every $X \cap P_k$ is computable by Proposition 3.1; a finite union of computable sections is computable. There need not be a computable sequence of indices for those sections. This does not obstruct the theorem: the construction decides finite admissibility with $H = X$, and the ordinary computable section indices are used only as finite constants in a fixed-stage decoding proof. The ideal is Comp , which is contained in every core.

Theorem 7.1 (Optimal first jump and joint lowness). *For the prescribed guarded c.e. set X there is a countable sequence (D_i) of binary sets, and $Z = \bigoplus_i D_i$, such that*

$$D_i \approx X \quad \text{for all } i; \quad (7.2)$$

$$\mathcal{L}(Z) \cap \mathcal{L}(\emptyset') = \text{Comp}; \quad (7.3)$$

$$D'_i \equiv_T Z' \equiv_T (Z \oplus \emptyset')' \equiv_T \emptyset'' \quad \text{for all } i. \quad (7.4)$$

Each $D_i \leq_T \emptyset''$ but $D_i \not\leq_T \emptyset'$. For nonempty finite S, T ,

$$\mathcal{L}(W_S) \cap \mathcal{L}(W_T) = \begin{cases} \text{Comp}, & S \cap T = \emptyset, \\ \mathcal{L}(W_{S \cap T}), & S \cap T \neq \emptyset. \end{cases} \quad (7.5)$$

Every nonempty finite join W_S also has jump $\emptyset''$, is low over $\emptyset'$, and forms a minimal pair with $\emptyset'$. The degree of Z is realized by a literal coarse description of X .

Proof. The data (7.1) satisfy the array theorem. It gives $D_i \approx X$, the exact common lower cone (7.3), and

$$(Z \oplus \emptyset')' \equiv_T \emptyset''.$$

Since $D_i \leq_T Z \leq_T Z \oplus \emptyset'$, monotonicity gives

$$D'_i \leq_T Z' \leq_T (Z \oplus \emptyset')' \equiv_T \emptyset''. \quad (7.6)$$

The opposite inequality $\emptyset'' \leq_T D'_i$ holds for every description of X by Lemma 3.2, or equivalently Theorem 3.5. Hence all inequalities in (7.6) are equalities. The construction yields $Z \leq_T \emptyset''$, and therefore $D_i \leq_T \emptyset''$.

The array theorem already ensures $D_i \not\leq_T \emptyset'$. Alternatively, if $D_i \leq_T \emptyset'$, then (7.3) and $D_i \leq_T Z$ would make D_i computable, contradicting Proposition 3.1. In particular each D_i is noncomputable, so the common-lower-cone statement with $\emptyset'$ is genuinely a minimal-pair assertion.

The finite-join intersection formula is (5.6) with $B_k = \emptyset$. For a nonempty S , choose $i \in S$. Then

$$\emptyset'' \equiv_T D'_i \leq_T W'_S \leq_T Z' \equiv_T \emptyset''.$$

Similarly $\emptyset'' \leq_T (W_S \oplus \emptyset')' \leq_T (Z \oplus \emptyset')' \equiv_T \emptyset''$. A common lower bound of W_S and $\emptyset'$ is a common lower bound of Z and $\emptyset'$, hence computable. Finally apply Corollary 5.3 to realize the joined degree inside $\text{CD}(X)$. $\square$

7.1 In what sense the bound is optimal

The plan's first proposed upper bound was $D \leq_T \emptyset''$. The question whether $D' \leq_T \emptyset''$ remained separate. Theorem 7.1 answers it affirmatively for the prescribed X , and strengthens it to

$$D \approx X, \quad \mathcal{L}(D) \cap \mathcal{L}(X) = \text{Comp}, \quad D' \equiv_T (D \oplus X)' \equiv_T \emptyset''. \quad (7.7)$$

The first-jump lower bound is unavoidable: Theorem 3.5 gives $\emptyset'' \leq_T D'$ for every description, whether or not it is an exact partner. Therefore no first jump strictly below $\emptyset''$ can work. This is an exact optimality statement, not merely an upper bound whose sharpness is guessed from the method.

At the level of the set itself, the exact partner cannot lie below $\emptyset'$, as the proof shows. The assertions do not classify all degrees strictly between the two jumps, and they do not claim a least degree among the exact partners. They also do not say that every description of X is outside Δ_2^0 : the set X itself is a Δ_2^0 description. It is the *exactness with X* that excludes such a bound for our partner.

7.2 A single nonzero coarse class with substantial Turing structure

The class of X contains X of degree $\emptyset'$, the mutually independent D_i , all their nonempty finite joins, and a representative of the entire joined degree. At the Turing level, nonempty finite subsets S correspond injectively to $\deg_T(W_S)$; disjoint subsets give minimal pairs and intersecting subsets have exactly the shared join as their meet. At the coarse level all these representatives are equivalent.

There is no contradiction in two high degrees having computable meet. A common jump lower bound is not a common lower bound of the original degrees. In this construction all nonempty finite joins have the same jump $\emptyset''$, but their original degrees retain the finite-subset order pattern.

Corollary 7.2 (A familiar negative answer, with stronger witnesses). *Neither the uniform nor the nonuniform coarse class of X has a least Turing degree.*

Proof. If a least degree existed, it would be below both $\deg_T(X) = \emptyset'$ and $\deg_T(D_0)$. Their minimal-pair property would make it computable. Lemma 2.3 would then give a computable coarse description of X , contrary to Proposition 3.1. $\square$

This corollary is not presented as new: the bare failure of a least representative is the retired target C1. Its role is to show precisely what the stronger jump-controlled witnesses add to that known negative answer.

8 Examples, presentation dependence, and boundaries

8.1 The arithmetic ideal as a common-information core

Example 8.1 (All arithmetical sets, but no more). Take $A_i = \emptyset^{(i)}$, let $H = \bigoplus_i \emptyset^{(i)}$, and define F by (6.1). Then

$$\text{Core}(F) = \{A : (\exists n) A \leq_T \emptyset^{(n)}\}.$$

Thus the information shared by all descriptions of F is exactly the ideal of arithmetical sets, even though $F \equiv_T H$ computes the entire finite-jump sequence uniformly. The ideal is nonprincipal because the finite jumps strictly increase. There are exact partners of F in $\text{CD}(F)$ whose join with

H has jump H' , and disjoint finite joins of the constructed descriptions have no greatest common lower degree.

This example uses the explicit join of finite jumps; no claim about ordinal notations, admissibility, or a transfinite jump hierarchy is required.

8.2 The same core can occur with or without a least degree

Example 8.2 (Two presentations of the computable ideal). If $A_i = \emptyset$ for every i , the profile F is computable, and its spectrum has least degree $\mathbf{0}$. Now let $K \subseteq \mathbb{N}$ be arbitrary and put

$$A_i = \begin{cases} \mathbb{N}, & K(i) = 1, \\ \emptyset, & K(i) = 0. \end{cases}$$

Every A_i is individually computable, and every finite join is computable. But the full join has degree $\deg_T(K)$, and the profile is exactly $F = \mathcal{R}(K)$ because the block code of a constant set is constant. Theorem 6.2 gives $\text{Core}(F) = \text{Comp}$, while Proposition 2.6 gives spectrum

$$\{\mathbf{b} : \deg_T(K) \leq \mathbf{b}'\}.$$

For $K = \text{TOT}$, this spectrum has no least degree: apply Theorem 6.2 to obtain an exact pair for Comp in the description class, and note that $\mathcal{R}(\text{TOT})$ has no computable description by Proposition 2.6.

There is no contradiction between the individual computability of every A_i and the possible noncomputability of their full join. The missing information is a uniform selection of their characteristic-function indices. This example is also a direct warning against replacing the presentation oracle H by an alleged computable list of local programs.

More generally, a constant profile $A_i = A$ has $F \equiv_T A$ and core $\mathcal{L}(A)$, so its spectrum has least degree $\deg_T(A)$. This is a fact about that particular uniform presentation, not about every profile generating the same principal ideal.

8.3 What the density proof does and does not control

The construction ensures, for every fixed i, k ,

$$D_i \triangle F \subseteq [0, \ell_{i,k}) \cup Q_k.$$

In the dyadic applications this yields

$$\rho_N(D_i \triangle F) \leq \ell_{i,k}/N + 2^{-k}.$$

The cutoffs $\ell_{i,k}$ are computable in H' from the construction, but need not be computable without it. In particular, these estimates do not give an arbitrarily prescribed computable pointwise bound on the number of disagreements. They do not establish target C7, whose sparsity budget and 1-genericity requirements are different.

A perfect family with every branch below one fixed oracle is also not asserted. There are only countably many sets below H' . The present theorem deliberately constructs a countable array; the reservoir synthesis's perfect-tree statements attach the branch parameter to their complexity bounds [6].

8.4 Why effective-dense reducibility is not substituted

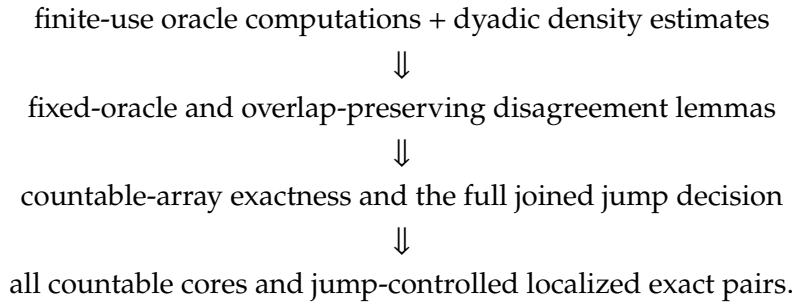
An effective-dense description may explicitly omit a density-zero set of answers but may not give an incorrect answer anywhere. This is different from coarse description, which tolerates wrong values on a negligible set; see the systematic comparison in Astor–Hirschfeldt–Jockusch [3].

Our construction intentionally changes bits on newly selected free positions, and later protection leaves those finite errors untouched. Neither a description itself nor a reduction using it is supplied with a computable list of the erroneous positions. Therefore one cannot turn the present argument into an effective-dense proof by declaring its mistakes to be omissions. The resulting statement would require a new hypothesis or a new construction. In particular, the results here leave C2 open as a target of the repository plan.

9 Verification, effective content, and formalization

9.1 Logical dependencies and the oracle budget

The mathematical dependency chain is explicit:



The optimal lower bound in C6 follows from the separate totality-decoding chain of Section 3. No category theorem, minimal-degree existence theorem, or jump-inversion theorem is invoked to obtain the main constructions.

Operation	Oracle used	Reason
Check a finite admissible array	H	P_k is uniformly H -computable and $F \leq_T H$.
Ask for a disagreement witness	H'	Existence is $\Sigma_1^0(H)$.
Find a positive witness	H	Enumerate finite arrays and finite computations.
Compute a common output after a negative decision	B_k , or $B_k \oplus W_U$	The condition and local program indices are fixed finite constants.
Decide $(Z \oplus H)'$	H'	Replay the stage making its permanent halting decision.
Obtain the density limit	No additional oracle assertion	Fix k , take the limsup in N , then let k grow.

None of these operations asks whether a functional is total. The formulas in the disagreement tests search for two convergent, unequal values. This is the reason the upper bound remains at H' rather than rising to H'' .

9.2 A proposed implementation boundary

The repository’s coverage ledger distinguishes actual degree-order infrastructure from existence theorems and from statements requiring explicit computational hypotheses [7]. This article does not claim that the new reservoir construction has been implemented or checked in Lean or Rocq. A faithful formalization would require the following layers.

Finite combinatorics. Define the dyadic pairing and prove the exact cardinality formulas. Represent finite array conditions by finite maps from coordinate indices to binary lists. Prove that admissibility is preserved under compatible amalgamation and that raising the protection level shrinks the reservoir. This layer is independent of jumps.

Oracle computation. Use a concrete model with a verified finite-use theorem. Encode finite joins and the dyadic countable join. Prove that a convergent oracle computation is witnessed by a finite condition, including computations whose oracle is the join with the fixed H .

Effective decisions. Express the three positive tests as H -computably enumerable predicates, with explicit index transformations. A general jump interface must deliver both the yes/no decision and a verified witness search in the positive case. Local indices witnessing B_k -computability should remain existential parameters in the negative-case proof; they should not silently become a uniformly computable function of k .

Construction and semantics. Define the deterministic H' -recursive stage sequence, prove totality of the coordinate limits, and show that the limit belongs to every previous reservoir. Prove the recorded jump bits equivalent to actual halting, not just sound in the positive direction. The overlap proof must use compatible shared coordinates. Only after these steps should the construction be projected to Turing degrees and coarse classes.

A file containing assumed lemmas with the same names would not certify the theorem. In particular, postulating the fixed-oracle exact partner or the full jump bound would remove the principal mathematical content.

9.3 Finite diagnostic checks included in the package

The accompanying standard-library Python program checks exact dyadic counts, the block-majority error inequality, preservation of finite reservoir conditions, and a finite truth-table version of overlap-preserving amalgamation. The shared-coordinate diagnostic also checks a counterexample to treating shared coordinates as independent. The checks test the finite combinatorics used by the proofs; they cannot verify oracle-jump decisions, infinite convergence, or the Turing-degree assertions.

The source is compiled independently of the checks. A successful PDF build validates document production, not the mathematical assertions. The manuscript includes full conventional proofs so that their validity can be assessed separately from the diagnostics and the typesetting.

[write] In report 13 the program is `code/finite_checks.py` and its recorded run is `data/results.json`. The program writes `results.json` beside itself, so it should be run on a copy (see the report’s `README.md`). At intake a run on a copy passed and reproduced the recorded file apart from line endings.

10 Further research questions and a prioritized programme

The following questions are stated as directions left by this article. Some are repository targets; others are refinements motivated by the proved results. Except where a source target is identified, no claim is made that the exact formulation is an established literature-listed open problem.

Question 10.1 (Prescribed generic targets: the remaining part of C6). Let $G \leq_T \emptyset'$ be a prescribed 1-generic set. Is there an arithmetical $D \approx G$ such that $\mathcal{L}(D) \cap \mathcal{L}(G) = \text{Comp}$? Can a bound be expressed using finitely many jumps of G ?

The present theorem requires increasingly large regions on which the target is computable in suitable local oracles. The guarded c.e. set has exactly that structure; a general 1-generic target is not furnished with it. A useful next step is to isolate a replacement for local decidability of admissibility which still makes the no-disagreement search compute an ordinary computable output.

Question 10.2 (Sparse generic minimal pairs below the first jump: C7). For every computable nondecreasing unbounded $h : \mathbb{N} \rightarrow \mathbb{N}$, can one obtain 1-generic $A, B \leq_T \emptyset'$ forming a minimal pair with

$$|(A \triangle B) \cap [0, n]| \leq h(n)$$

for all n ?

The present construction neither proves 1-genericity nor meets an arbitrary counting budget. Its reservoir tail bounds concern density rather than an a priori finite-error allowance. The repository's one-bit-reserve arguments are a more directly relevant starting point [5, 6].

Question 10.3 (Intrinsic hypotheses for bounded exact partners). Which conditions on $F \leq_T H$ imply that there is $D \approx F$ with

$$\mathcal{L}(D) \cap \mathcal{L}(H) = \text{Core}(F), \quad (D \oplus H)' \equiv_T H'?$$

Can the layered-presentation hypotheses of Theorem 5.1 be replaced by a condition stated solely in terms of the coarse description mass problem?

The distinction between the target and its presentation is substantial: the local programs may exist nonuniformly. A promising formulation should preserve a small-oracle search for common values while allowing H to decide admissibility effectively.

Question 10.4 (The spectrum of a general robust profile). For $F(c(i, t)) = J(A_i)(t)$, what additional information about the sequence (A_i) , beyond its generated ideal, determines the full coarse-description spectrum or the uniform coarse degree of F ?

Example 8.2 shows that the core alone is insufficient: the same computable ideal occurs in a computable class and in a class with the unrestricted high spectrum. One can seek a classification in terms of uniform versus nonuniform access to the finite joins of the profile.

Question 10.5 (Minimal elements, not only least elements: C5). When does a coarse-description spectrum have a minimal Turing degree even though it has no least degree? Can the ideal and the presentation together provide an exact criterion for robust column profiles?

The constructions here produce many incomparable representatives and exact pairs, but that does not decide whether some other representative is minimal in the spectrum. A proof ruling out a least degree must not be promoted to a proof ruling out every minimal element.

Question 10.6 (Density-modulus costs). How does the smallest possible exact-partner jump change when a reduction is given a modulus for the density-zero error, or when the partner must satisfy a prescribed computable modulus?

For the present dyadic examples, the cutoffs $\ell_{i,k}$ provide a modulus relative to the construction oracle. Making them computable may reveal information which was hidden in the nonuniform local programs. A first bounded problem is to characterize which moduli allow uniform recovery of the profile oracle from a description.

Question 10.7 (Beyond finite coordinate sets). Given a uniformly computable family of infinite coordinate sets $S_m \subseteq \mathbb{N}$, can one impose analogous exact intersection identities on the subjoins $\bigoplus_{i \in S_m} D_i$ while retaining $(Z \oplus H)' \equiv_T H'$?

Finite-use computations still inspect only finitely many coordinates, suggesting that the overlap lemma may extend to suitable uniformly presented families. The statement must specify how each subjoin is encoded and how membership in the shared coordinate set is accessed; a nonuniform family of arbitrary masks introduces new information.

Question 10.8 (Effective-dense least representatives: C2). Does every nonuniform effective-dense equivalence class have a representative of least Turing degree? Which versions survive when all functions are binary-valued?

Wrong answers cannot be used as omissions. A solution requires either an omission-aware analogue of the reservoir construction or a structural reason why a least representative always exists. The distinctions formalized in [3] are indispensable.

Question 10.9 (Relative guarded universality). For an oracle V , define the guarded set using (φ_e^V) in place of (φ_e) . What is the sharp analogue of Theorem 3.5 when coarse reductions themselves are allowed the fixed oracle V ? What remains true for ordinary, unrelativized coarse reductions between those same sets?

The first question is a natural relativization of the displayed algorithms. The second is different: an ordinary reduction may not call V to simulate the guarded programs. Any theorem should state explicitly which oracle is available to the transformation, not only to the construction of the target.

Question 10.10 (A verified countable-array forcing library). Can the entire proof of Theorem 5.1, including its overlap and joined-jump clauses, be implemented in the repository's chosen proof assistant without adding an exact-pair axiom or assuming the jump bound?

The smallest useful milestone is the finite witness and amalgamation API. The next is a verified translation of admissible-witness searches into $\Sigma_1^0(H)$ indices. Once these are available, the same infrastructure should support several existing reservoir and degree-construction projects.

A practical order of work is to formalize the finite-array lemmas first, analyze the uniform spectrum of a robust profile next, and only then attack the generic and effective-dense targets. These priorities concern dependency structure, not claims about how long a problem must take.

11 Conclusion

The local computability of increasingly large protected regions has two distinct consequences. It permits H' -effective finite decisions, and it ensures that a failed disagreement search yields an

output computable in a smaller stage oracle. Separating those consequences gives a countable-array theorem with exact ideal intersections and a genuine bound on the jump of the full join.

Applied to robust column profiles, this proves the exact C4 formula and characterizes all common-information cores as the countable Turing ideals. Applied to the prescribed guarded c.e. set, and combined with the explicit equivalence to $\mathcal{R}(\text{TOT})$, it gives exact partners with an unavoidable and attained first jump $0''$. The countable family retains a finite-subset pattern of ordinary Turing joins and meets while lying in one uniform coarse class.

The central gain over a bare existence argument is therefore simultaneous control: locality in one density-zero class, exactness against a prescribed oracle, finite-join intersections, and one optimal joined jump bound. The remaining questions concern the hypotheses needed for this control and the changes required for genericity or omission-only descriptions.

A An explicit proof audit

The following audit identifies the points at which an apparently similar argument can prove a different or false statement.

Nonuniform section indices. The guard proves that each section is computable by a case distinction. It does not give a computable map from the section number to a program. Construction-time admissibility uses H , where membership in the target is decidable; verification-time decoding fixes one section level and hard-codes its ordinary local program indices. No uniform selector is inferred.

Single-array overlap. The test for finite-join disagreement uses one finite array. Shared coordinates must agree between the two computations. In the negative-case proof, candidate values on the shared coordinates are checked against the actual shared oracle before amalgamation. Without this restriction, one can falsely diagonalize a common output that is simply a shared coordinate.

Persistent negative decisions. A negative jump or disagreement decision quantifies over every admissible finite extension of the current condition. Every later reservoir is a subset of the current one, even when its protection level increases. This is why a decision made once remains valid.

Finite mistakes under stronger protection. Raising the protection level does not reset old stem bits. Their possible errors form a finite initial segment for each fixed coordinate and stage, producing the term $\ell_{i,k}/N$ in the density estimate. A construction that reset these bits would no longer be a monotone finite-extension construction.

Countable union of errors. The proof does not use countable additivity of asymptotic density. It fixes finitely many positive-density columns and bounds the remaining tail uniformly by 2^{-k} . This distinction is necessary both for the limit code and for countable repetition.

Totality only in verification. The searches ask for finite witnesses of convergence or inequality. No stage decides whether an oracle program is total. Common-output computability is proved under the explicit hypothesis that both reductions actually compute the same total function.

The whole joined jump. The jump requirements refer to $Z \oplus H$ itself. A list of separately bounded column jumps would not imply the claimed joined bound. The recorded answers are proved equivalent to actual halting using finite-use witnesses in both directions.

Optimality has an independent proof. The upper bound comes from the construction. The lower bound for C6 comes from totality coding in specially chosen columns of the prescribed X . It applies to every description of that exact set. The conclusion is not generalized to every complete c.e. set.

Principal ideals and least degrees. A least spectrum degree implies a principal core, but the converse fails. In a profile of individually computable sets, their full join may still be noncomputable because the presentation is not uniform. Example 8.2 makes this failure explicit.

Verification status. The article contains conventional proofs, a source comparison, and executed finite diagnostics. No Lean or Rocq build of these new results has been performed, and no independent referee assessment is represented. The precise novelty claims are relative to the inspected plan and synthesis, not to an exhaustive search of all computability literature.

B Source provenance and reproducibility

The repository sources used for the mathematical target and dependency comparison are the research plan, the round-one synthesis, and the coverage ledger [5, 6, 7]. The source-file identities were checked at the commit printed in Section 1. The relevant plan source blob is

```
ebecd2e15f8e4a06ebd0b0333387d27ebe630375,
```

and the synthesis source blob is

```
c88062125f64aa3e9323f3d3f39f5f7e108a6a7a.
```

The public literature check included the original Jockusch–Schupp paper, the Hirschfeldt–Jockusch–Kuyper–Schupp paper and author page, and primary publication records for the related works listed below. The arXiv version of Jockusch–Schupp inspected here is the 2010 preprint; its relevant dyadic result is also identified by theorem number.

The archive contains `article.tex`, the compiled `article.pdf`, a build script, finite checks, their result file, and separate source and proof-audit records. The bibliography is embedded in the TeX source. The PDF can be rebuilt with

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

and the diagnostics with

```
python3 checks/finite_checks.py
```

No external figures or distributed font binaries are required.

[write] Report 13 ships `article.tex` and a build of it as `article.pdf`, not the delivered 26-page PDF. The diagnostics are `code/finite_checks.py` with the recorded run `data/results.json`; the audit records are `PROOF_AUDIT.md`, `SOURCES.md` and `data/validation.json` (the last describes the delivered PDF). The delivered `build.sh`, shipped as `code/build.sh`, changes to its own directory and runs `checks/finite_checks.py` and `latexmk` there, so it does not work in the shipped layout; the report’s `README.md` gives working commands.

References

- [1] C. G. Jockusch, Jr. and P. E. Schupp, *Generic computability, Turing degrees, and asymptotic density*, Journal of the London Mathematical Society (2) **85** (2012), 472–490. doi:10.1112/jlms/jdr051. Preprint: <https://arxiv.org/abs/1010.5212>.
- [2] D. R. Hirschfeldt, C. G. Jockusch, Jr., R. Kuyper, and P. E. Schupp, *Coarse reducibility and algorithmic randomness*, Journal of Symbolic Logic **81** (2016), 1028–1046. doi:10.1017/jsl.2015.70. Preprint: <https://arxiv.org/abs/1505.01707>.
- [3] E. P. Astor, D. R. Hirschfeldt, and C. G. Jockusch, Jr., *Dense computability, upper cones, and minimal pairs*, Computability **8** (2019), 155–177. doi:10.3233/COM-180231. Preprint: <https://arxiv.org/abs/1811.07172>.
- [4] G. Barmpalias and R. Downey, *Exact pairs for the ideal of the K-trivial sequences in the Turing degrees*, Journal of Symbolic Logic **79** (2014), 676–692. doi:10.1017/jsl.2014.37.
- [5] ProveIt repository, *Turing Degrees: A Unified Research Report*, second edition, 18 September 2026, research plan and subsequent repository revisions. Source path: `Computability/TuringDegrees/Research/CoarseDegrees/research-plan/turing_degrees_unified.tex`. C4 and C6 are the targets used here. Commit-pinned source in ProveIt.
- [6] ProveIt repository, *Coarse Classes Without Least Turing Degrees: Deduplicated results of the nine research reports on target C1*, 18 September 2026. Source path: `Computability/TuringDegrees/Research/CoarseDegrees/research-synthesis/Turing_Degrees_Synthesis.tex`. The guarded diagonal set and reservoir constructions are used as the starting point, with their needed proofs supplied in the present article. Commit-pinned source in ProveIt.
- [7] ProveIt repository, *Coverage of the Turing-degree article*. Source path: `Computability/TuringDegrees/COVERAGE.md`. Commit-pinned coverage ledger.